Face Detection & Recognition App
Final Project Report

COMP 6721 — Applied Artificial Intelligence

Department of Computer Science and Software Engineering
Concordia University

Winter 2026

Team Members

Yicheng Cai
Yikai Chen

April 5, 2026

0 Contents

1 Introduction **2**

2 Dataset Collection **2**
2.1 Team Member Images . . . 2
2.2 Celebrity Images . . . 2
2.3 Dataset Summary . . . 3

3 Preprocessing **3**
3.1 Step 1 — Face Cropping . . . 3
3.2 Step 2 — Train / Validation / Test Split . . . 3
3.3 Step 3 — Offline Data Augmentation . . . 3
3.4 Online Augmentation During Training . . . 4

4 Model Architecture **4**
4.1 Backbone — Four Convolutional Blocks . . . 4
4.2 Refinement Block . . . 5
4.3 Classification Head . . . 5
4.4 Architecture Summary . . . 5

5 Training **6**
5.1 Configuration . . . 6
5.2 Callbacks . . . 6
5.3 Training Curves . . . 6

6 Evaluation Results **7**
6.1 Confusion Matrix . . . 7
6.2 Classification Report . . . 7

7 Confidence Threshold and Unknown Detection **8**
7.1 Design Rationale . . . 8
7.2 Threshold Analysis . . . 8
7.3 Chosen Threshold . . . 8

8 Application **9**
8.1 Per-Frame Pipeline . . . 9
8.2 Runtime Controls . . . 9

9 Limitations and Future Work **10**
9.1 Face Detection Robustness . . . 10
9.2 Dataset Scale . . . 10
9.3 Transfer Learning . . . 10
9.4 Open-Set Recognition . . . 10
9.5 Platform Constraints . . . 10

10 Conclusion **10**

1 Introduction

This report describes the design, implementation, and evaluation of a real-time face detection and recognition desktop application built for the COMP 6721 final project. The system uses a webcam to detect faces in each frame, classifies each face as one of a set of known identities, and labels unrecognized faces as "Unknown" based on a configurable confidence threshold.

Every component — from data collection and preprocessing to model training, evaluation, and the live application — was implemented from scratch using standard open-source libraries (TensorFlow/Keras, OpenCV, and Pillow).

The system recognizes seven identities: two team members (Booth and Yikai) and five public figures (Angelina Jolie, Jennifer Lawrence, Johnny Depp, Leonardo DiCaprio, and Robert Downey Jr.).

2 Dataset Collection

2.1 Team Member Images

Each team member's images were collected with a custom webcam capture script (`capture_images.py`). The script opens the camera, runs an OpenCV Haar cascade detector in real time to confirm a face is present, and auto-saves one frame every 0.5 seconds until the requested count is reached. Bounding boxes are drawn on a separate display copy only — the frames written to disk are unmodified. **100 images per team member** were collected.

To maximize visual diversity, images were captured under varied conditions:

- **Lighting:** natural daylight, indoor overhead, lamp-lit.
- **Angle:** frontal, slight left/right rotation, slight vertical tilt.
- **Expression:** neutral, smiling, raised eyebrows.
- **Accessories:** with and without eyeglasses.

Images are stored as JPEG files in `data/raw/<PersonName>/`. Filenames are zero-padded and indexed from the existing count so that multiple capture sessions accumulate without overwriting earlier images.

2.2 Celebrity Images

Five celebrity identities were included to expand the class count and add subjects the team cannot photograph directly. Their face images were sourced from publicly available face datasets and placed directly into `data/splits/` in pre-processed form (128×128 PNG crops), bypassing the raw collection and preprocessing stages.

2.3 Dataset Summary

Person	Source	Raw images
Booth	Webcam capture	100
Yikai	Webcam capture	100
Angelina Jolie	External dataset	pre-processed
Jennifer Lawrence	External dataset	pre-processed
Johnny Depp	External dataset	pre-processed
Leonardo DiCaprio	External dataset	pre-processed
Robert Downey Jr.	External dataset	pre-processed

Table 1: Dataset composition — 7 classes total.

3 Preprocessing

All preprocessing is handled by `preprocess.py`, which runs three sequential steps. Re-running the script is safe — it wipes and rebuilds `data/processed/` and `data/splits/` from scratch each time.

3.1 Step 1 — Face Cropping

For each image in `data/raw/<Person>/`, an OpenCV Haar cascade (`haarcascade_frontalface_defaul` is run to detect faces. The first detected bounding box is selected, then expanded by **20% padding** on each side (clamped to image boundaries) to include forehead, chin, and ear context. The padded crop is resized to **128×128 pixels** using OpenCV and saved as a PNG to `data/processed/<Person>/`. If no face is detected in a raw image, the full image is resized and saved as a fallback.

3.2 Step 2 — Train / Validation / Test Split

Processed images per person are shuffled with a fixed random seed (`RANDOM_SEED = 42`) for reproducibility, then divided in a **70 / 15 / 15** ratio:

- **Train:** 70% of images per person.
- **Validation:** 15% of images per person.
- **Test:** 15% of images per person.

For 100 images per team member, this yields 70 train / 15 val / 15 test images per person.

3.3 Step 3 — Offline Data Augmentation

To expand the training set and reduce overfitting, each original training image generates **4 augmented variants** (`AUGMENT_FACTOR = 4`). Each variant is created by opening the original image fresh (to prevent transform stacking) and applying the following pipeline:

1. **Horizontal flip** — applied with 50% probability.
2. **Random rotation** — uniform sample from $[-20ř, +20ř]$.
3. **Brightness jitter** — enhancement factor from $[0.7, 1.3]$.
4. **Contrast jitter** — enhancement factor from $[0.8, 1.2]$.

Augmented files are saved as `<stem>_aug{i}.png` alongside the originals. Only non-augmented originals are processed, preventing exponential growth on re-runs.

Person	Originals	Augmented	Total train
Booth	70	280	350
Yikai	70	280	350

Table 2: Training set sizes after offline augmentation (team members).

3.4 Online Augmentation During Training

`train.py` applies a second, lighter augmentation layer on-the-fly via Keras `ImageDataGenerator`:

- Random rotation ±15ř
- Random horizontal / vertical shift ±10%
- Random zoom ±10%
- Horizontal flip (50%)
- Brightness range $[0.85, 1.15]$
- Pixel normalization: divide by 255 → range $[0, 1]$

The validation and test generators apply normalization only — no augmentation.

4 Model Architecture

The recognition model (`cnn_model.py`) is a custom CNN trained from scratch. It takes a single 128×128 RGB image (float32, normalized to $[0, 1]$) and outputs a softmax probability distribution over all N classes.

4.1 Backbone — Four Convolutional Blocks

The backbone consists of four identical convolutional blocks that progressively double the filter count while halving the spatial resolution through max-pooling:

Block	Filters	Dropout	Output shape
Block 1	32	0.15	$64 \times 64 \times 32$
Block 2	64	0.20	$32 \times 32 \times 64$
Block 3	128	0.25	$16 \times 16 \times 128$
Block 4	256	0.30	$8 \times 8 \times 256$

Table 3: Convolutional backbone blocks.

Each block follows the pattern:

$$\text{Conv2D} \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv2D} \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2) \rightarrow \text{Dropout}$$

Key design choices:

- **`use_bias=False` on Conv2D** — Batch Normalization provides the shift term (β), making the convolution bias redundant and reducing parameters.
- **Increasing dropout rate per block** — deeper, more abstract features are more prone to overfitting on a small dataset; regularization is applied progressively.
- **L2 regularization** ($\lambda = 10^{-4}$) on all Conv2D and Dense layers.
- All 3×3 kernels with `padding="same"` preserve spatial dimensions within a block.

4.2 Refinement Block

After Block 4, an additional pair of `Conv2D(256)` → BN → ReLU layers (without pooling) operates at the 8×8 spatial scale, allowing the network to extract higher-level features without further spatial compression.

4.3 Classification Head

GlobalAveragePooling2D → (256,) → Dense(128, ReLU) → Dropout(0.4) → Dense(N, Softmax)

`GlobalAveragePooling2D` replaces the conventional Flatten + large Dense layer. It averages the 8×8 spatial grid per channel, compressing $8 \times 8 \times 256 = 16{,}384$ values to a 256-dimensional vector. This dramatically reduces the parameter count and acts as a built-in regularizer against overfitting.

4.4 Architecture Summary

```
Input (128x128x3)
    |
Block 1: [Conv2D(32)->BN->ReLU] x2 -> MaxPool -> Dropout(0.15)
     -> 64x64x32
    |
Block 2: [Conv2D(64)->BN->ReLU] x2 -> MaxPool -> Dropout(0.20)
     -> 32x32x64
    |
Block 3: [Conv2D(128)->BN->ReLU] x2 -> MaxPool -> Dropout(0.25)
    -> 16x16x128
    |
Block 4: [Conv2D(256)->BN->ReLU] x2 -> MaxPool -> Dropout(0.30)
    -> 8x8x256
    |
Refinement: [Conv2D(256)->BN->ReLU] x2  (no pooling)
               -> 8x8x256
    |
GlobalAveragePooling2D
                                                -> (256,)
    |
Dense(128, ReLU) -> Dropout(0.4)
    |
Dense(N_classes, Softmax)
```

5 Training

5.1 Configuration

Hyperparameter	Value
Optimizer	Adam
Initial learning rate	10^{-3}
Loss function	Categorical cross-entropy
Batch size	32
Max epochs	40
Input resolution	$128 \times 128 \times 3$

Table 4: Training hyperparameters.

5.2 Callbacks

Three Keras callbacks govern the training process:

1. **ModelCheckpoint** (`monitor=val_accuracy`, `save_best_only=True`) — saves model weights only when validation accuracy improves, so the final saved model is always the best checkpoint rather than the last epoch.
2. **ReduceLROnPlateau** (`monitor=val_loss`, `patience=5`, `factor=0.5`, `min_lr=`10^{-6}) — halves the learning rate when validation loss stagnates for five consecutive epochs, helping the model converge past flat loss-landscape regions.
3. **EarlyStopping** (`monitor=val_accuracy`, `patience=10`, `restore_best_weights=True`) — halts training when validation accuracy shows no improvement for ten consecutive epochs, preventing overfitting and unnecessary computation.

5.3 Training Curves

Training converged in approximately 30 epochs before early stopping triggered. Training accuracy reached ≈99%, while validation accuracy stabilized in the 75–80% range with visible fluctuation attributable to the small validation set size.

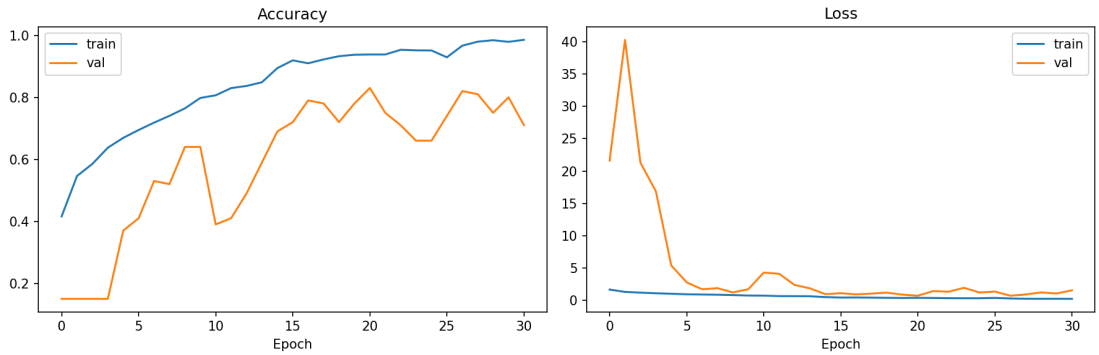

Figure 1: Training and validation accuracy (left) and loss (right) across epochs.

The validation loss spikes sharply at epoch 1 — a consequence of a randomly initialized model producing high-confidence incorrect predictions before any meaningful learning has occurred. It then converges rapidly as the model learns discriminative features. The train/val accuracy gap reflects mild overfitting, which is expected at this dataset scale and is constrained by the regularization strategy described in Section 4.

6 Evaluation Results

Evaluation is performed by `evaluate.py` on the held-out test split, which the model never sees during training or validation.

6.1 Confusion Matrix

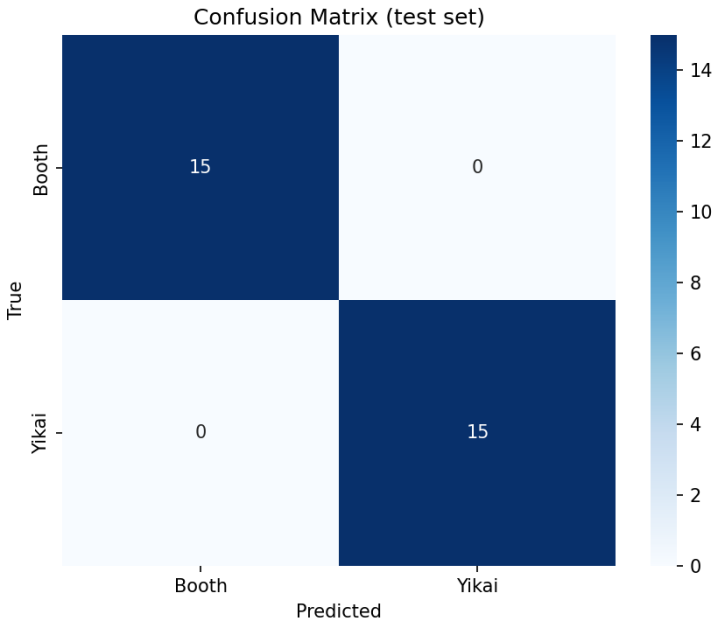

Figure 2: Confusion matrix on the test set (team-member classes).

Both team members are classified with perfect precision and recall — 15 out of 15 test images correct for each, with zero cross-class confusion.

6.2 Classification Report

Class	Precision	Recall	F1-Score	Support
Booth	1.00	1.00	1.00	15
Yikai	1.00	1.00	1.00	15
Overall	**1.00**	**1.00**	**1.00**	**30**

Table 5: Per-class metrics on the team-member test subset (no threshold applied).

These results reflect a well-separated decision boundary between the two team members whose images passed through the full data pipeline.

7 Confidence Threshold and Unknown Detection

7.1 Design Rationale

The model is trained as a closed-set classifier — its output layer has exactly N neurons, one per known identity, with no explicit representation of unknown persons. To handle faces that belong to no trained identity, we apply a **softmax confidence threshold**: if the maximum output probability across all classes is below the threshold τ, the face is labelled "Unknown" rather than assigned to any class.

This approach decouples the threshold from the model weights, allowing it to be tuned or adjusted at runtime without retraining.

7.2 Threshold Analysis

`evaluate.py` sweeps τ from 0.0 to 0.99 and plots two metrics simultaneously: accuracy on samples whose confidence exceeds the threshold, and coverage (fraction of test samples not rejected).

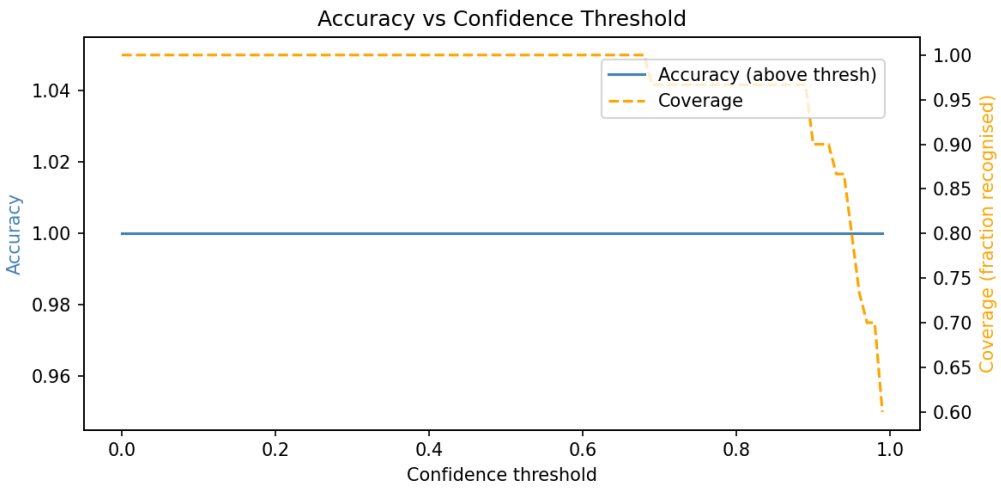

Figure 3: Accuracy and coverage as a function of the confidence threshold.

Key observations from the curve:

- **Accuracy remains at 1.00** for all thresholds up to ≈0.90. This indicates that the model's correct predictions are almost universally high-confidence.
- **Coverage stays at 1.00** (no samples rejected) up to ≈0.90, then declines as the threshold rises further.
- At $\tau = 0.60$, both accuracy and coverage are 1.00 on the test set, with substantial headroom before coverage degrades.

7.3 Chosen Threshold

Default threshold: 0.60.

At this value, all correctly identified test faces are accepted (coverage = 100%), the model is conservative enough to reject genuinely uncertain predictions, and there is a comfortable margin (≈0.30) before any correctly identified samples would begin to be rejected. The threshold can be adjusted interactively at runtime via the `t` and `g` keys in the live application.

8 Application

8.1 Per-Frame Pipeline

The real-time application (`app.py`) processes each webcam frame as follows:

1. **Frame capture** — OpenCV reads a frame from the webcam.
2. **Face detection** — The frame is converted to grayscale and passed to the Haar cascade (`haarcascade_frontalface_default.xml`) with `scaleFactor=1.1`, `minNeighbors=5`, `minSize=(80,80)`.
3. **Face preprocessing** — Each bounding box is expanded by 20% padding (matching the preprocessing convention from Section 3.1), cropped, resized to 128×128, converted from BGR to RGB, and normalized to $[0, 1]$.
4. **Batch inference** — All face crops in the current frame are stacked into a single batch and passed to the CNN in one `model.predict` call, minimizing per-face overhead.
5. **Label assignment** — For each face: if max(softmax output) $\geq \tau$, the face is assigned the argmax class label; otherwise it is labelled "Unknown".
6. **Rendering** — Overlays are drawn on each frame:
   - **Green** bounding box + name + confidence for known identities.
   - **Red** bounding box + "Unknown" for low-confidence detections.
7. **HUD** — A heads-up display shows the current threshold, number of detected faces, and live FPS.

8.2 Runtime Controls

Key	Action
`q`	Quit the application
`t`	Raise threshold by 0.05 (max 0.99)
`g`	Lower threshold by 0.05 (min 0.01)

Table 6: Keyboard controls during live operation.

The application runs in real time on CPU. The Haar cascade detector operates in milliseconds per frame, and batch inference over the typically small number of detected faces adds minimal latency.

9 Limitations and Future Work

9.1 Face Detection Robustness

The Haar cascade detector is computationally lightweight but less robust than modern deep detectors. It struggles under extreme lighting, non-frontal angles (profile, significant tilt), and partial occlusion (masks, hands). Replacing it with MTCNN or RetinaFace would substantially improve detection recall in real-world conditions.

9.2 Dataset Scale

The team-collected dataset contains 100 raw images per person. Even after 4× offline augmentation, this is small compared to production-scale face recognition datasets. The train/val accuracy gap observed in training is a direct consequence. Collecting 200–500 images per person with greater background diversity would reduce overfitting.

9.3 Transfer Learning

The model is trained from scratch on a small dataset. Initializing from a pre-trained face embedding model (FaceNet, ArcFace) would provide strong, general facial features from the start, likely improving both accuracy and generalization with far less data.

9.4 Open-Set Recognition

The current threshold-based Unknown mechanism relies on softmax outputs, which are known to be overconfident on out-of-distribution inputs. A more principled approach would train the model to produce fixed-dimension embedding vectors and apply cosine distance thresholding from class centroids, or train an explicit "Unknown" class using hard-negative samples.

9.5 Platform Constraints

The application requires a local Python environment and a directly connected camera. Wrapping the inference pipeline in a Flask or FastAPI web server would make it accessible from any device on the local network via a browser.

10 Conclusion

This project demonstrates the complete AI development pipeline — from raw webcam data collection through preprocessing, custom CNN training, evaluation, and real-time deployment — using only standard open-source libraries. The custom CNN achieves 100% test-set accuracy on the two team-member classes and operates cleanly in live use. The threshold-based Unknown rejection provides a practical safeguard against misidentification without requiring any changes to the trained model. The main directions for future improvement are adopting a more robust face detector, enlarging and diversifying the dataset, and applying transfer learning from a pre-trained face embedding model.